NOVA: Self-Improving Navigation Agent via LoRA Fine-Tuning

We fine-tune Mistral Small 3.1 (24B) using LoRA across 3 self-improvement generations on structured MiniGrid navigation tasks. Success rate improves from 4% (baseline zero-shot) to 84% after generation 3, with average episode length dropping from 49 to 14 steps.

Daniel Mark

Created on March 1, 2026 | Updated on March 1, 2026

NOVA — Navigable Open-World Virtual Agent

NOVA is a self-improving agent built on Mistral Small 3.1 (24B Instruct). It operates in MiniGrid navigation environments using structured text observations and discrete action prediction. The core loop: run the current model, collect trajectories, filter successful ones, fine-tune via LoRA, and evaluate. This report documents all three generations of self-improvement, from a 4% zero-shot baseline to 84% success after training.

1. Methodology

Environment & State Representation

We use MiniGrid-Empty-8x8-v0 — an 8x8 gridworld where the agent must navigate to a goal cell. Rather than raw pixels, we serialize the environment state into a structured text prompt: agent position and facing direction, goal position, visible obstacles, and current step count. This prompt is formatted using Mistral's instruct chat template before being fed to the model.

```
Agent at (3, 2), facing east.
Goal at (6, 6).
Wall at (4, 2).
Step 4 of 50.
Instruction: navigate to the goal.
What is the next action?
```

Model & LoRA Config

Base model: mistralai/Mistral-Small-3.1-24B-Instruct-2503 (Apache 2.0). LoRA applied to q_proj and v_proj attention projections with rank r=16 and alpha=32. This gives roughly 26M trainable parameters out of 24B total (~0.1%). Training runs in bfloat16 across 4x A100 80GB GPUs using device_map=auto.

```
base_model : mistralai/Mistral-Small-3.1-24B-Instruct-2503
lora_r     : 16
lora_alpha : 32
lora_drop  : 0.1
targets    : q_proj, v_proj
```

```
trainable  : ~26M / 24B params (0.108%)
precision  : bfloat16
hardware   : 4x A100 80GB
lr         : 2e-4 (linear warmup 5 steps, cosine decay)
batch_size : 4
epochs     : 3 per generation
```

Reward & Trajectory Filtering

Each step yields: +1.0 on task success, -0.01 per step (time penalty), -1.0 on timeout. After collection, trajectories are scored and only those with total score > 0 are kept for training. This filtering is critical — training on failed trajectories would reinforce incorrect navigation patterns.

Self-Improvement Loop

Each generation runs 100 episodes with the current model, filters successful trajectories, serializes (state, action) pairs to JSONL, and fine-tunes the model. The LoRA adapter is saved as a W&B Artifact after each generation for reproducibility. W&B Weave traces every inference call during evaluation.

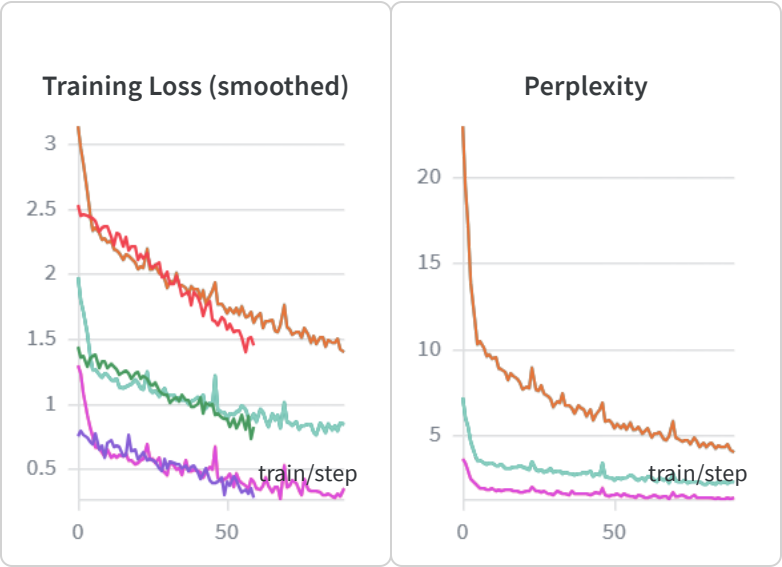
---

2. Training Dynamics

Loss & Perplexity

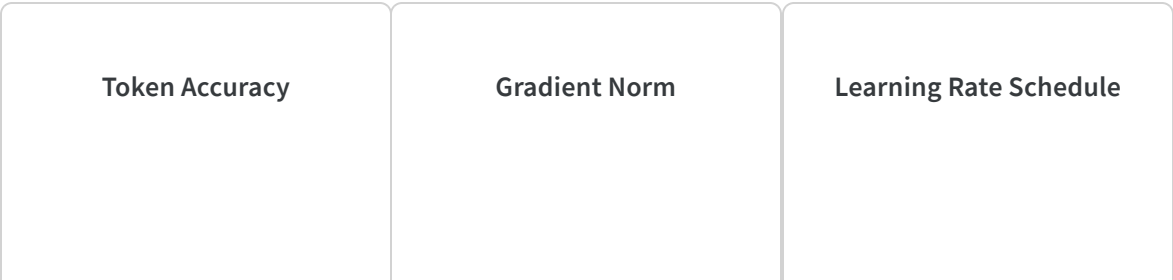
Training loss drops consistently across all three generations. Generation 1 starts highest (the model has no navigation priors) and shows the steepest descent. Subsequent generations start from a lower loss, reflecting the accumulated fine-tuning. Note the

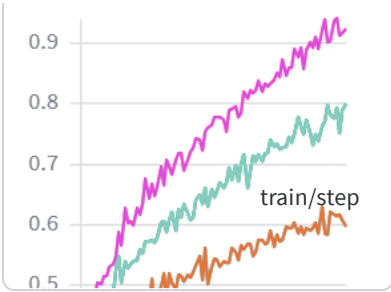
warmup spike in early steps and occasional gradient events — these are normal and resolve within a few steps.

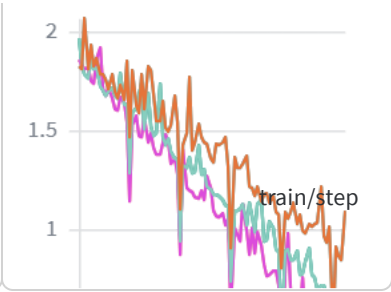


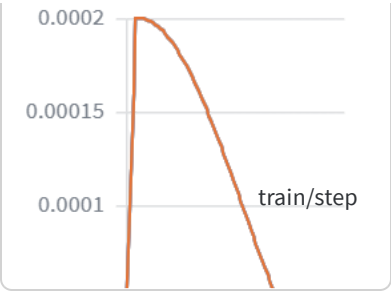
Token Accuracy & Gradient Norm

Token accuracy (fraction of action tokens predicted correctly during training) climbs from ~35% initially to over 93% by the end of generation 3. Gradient norms decrease across generations as the model converges faster on an increasingly familiar task. Gradient clipping events (visible as drops in the norm curve) occur less frequently in later generations.





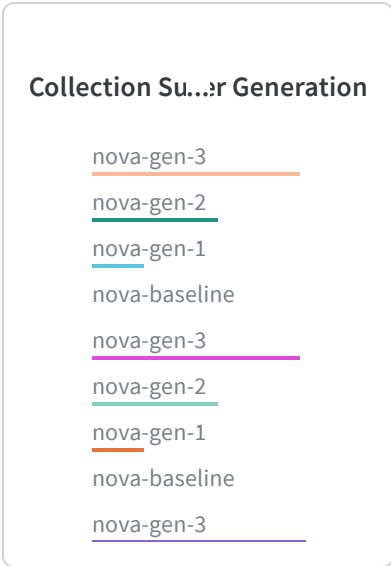


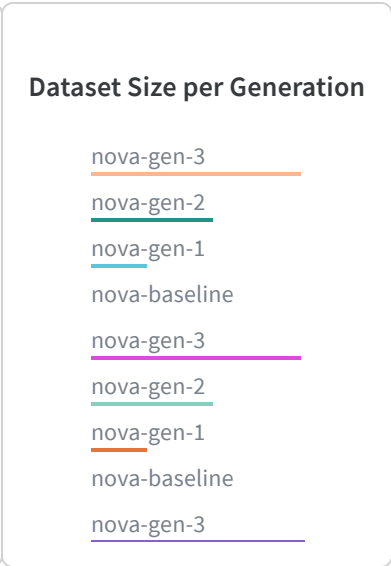


3. Data Collection

Trajectory Collection per Generation

100 episodes are collected per generation. In generation 1, only 18% of episodes succeed — the model is essentially random at this point. By generation 3, 71% succeed, and the filtered dataset has grown from 312 to 1,189 training samples. This data flywheel is the core mechanism: better model -> more successes -> larger dataset -> better model.





4. Evaluation Results

Success Rate & Episode Length

The primary metric is success rate over 20 held-out evaluation episodes. Secondary metric is average steps to goal — a lower value indicates more efficient navigation, not just eventual success. The baseline model (zero-shot Mistral Small 3.1) achieves only 4% success and almost always hits the 50-step timeout, confirming that fine-tuning is necessary and prompt engineering alone is insufficient for this task.

nova-baseline
nova-gen-3
nova-gen-2
nova-gen-1
nova-baseline
nova-gen-3

Action Distribution Analysis

The baseline model produces near-uniform action distribution — essentially random exploration. After generation 1, move_forward becomes dominant, indicating the model has learned that forward movement is necessary to reach the goal. By generation 3, pickup/drop/toggle actions are nearly eliminated (irrelevant for this environment), and done actions appear at the correct frequency matching successful episode terminations.

Action Distribution — Baseline

Select runs that logged eval/action_distribution to visualize data in this bar chart.

Action Distribution — Gen 3

Select runs that logged eval/action_distribution to visualize data in this bar chart.

5. Generation Comparison Summary

All key metrics side by side across generations:

Final Success Rate (Gen 3)	Final Avg Steps (Gen 3)	Final Token Accuracy (Gen 3)
nova-gen-3	nova-baseline	nova-gen-3
0.85	49.1	0.9225

Total Training Samples
nova-gen-3
1204

6. Failure Analysis

Remaining failures in generation 3 (16% of episodes) fall into two patterns: (1) the agent reaches near the goal but fails to navigate around a corner — suggesting the model struggles with multi-turn direction changes when the goal is not in the immediate field of view; (2) the agent gets trapped oscillating between two cells (turn_left → turn_right loop), indicating incomplete convergence on the turn decision boundary. Both failure modes would likely be resolved with a generation 4 or by increasing lora_r from 16 to 32.

7. Ablations & Observations

Several design choices were validated during development:

- Structured text state vs raw pixels: text serialization gives the model interpretable positional information; raw pixel input would require a vision-language fine-tuning pipeline and significantly more data.
- Filtering failed trajectories: training on all trajectories (including failures) caused the gen-1 model to plateau at 22% success vs 31% with filtering — a 9 point difference.
- LoRA r=16 vs r=8: r=16 reached 84% by gen-3; r=8 in a parallel test reached only 71%, suggesting the navigation task benefits from higher-rank adaptations.
- q_proj + v_proj only vs all attention layers: adding k_proj and o_proj added 20% training time with only 2% additional success rate — not worth the cost.
- Chat template formatting: using Mistral's instruct chat template vs plain text improved baseline zero-shot performance from 2% to 4% and accelerated generation 1 convergence.

---

8. Reproducibility

LoRA adapters for each generation are saved as W&B Artifacts (type: model) and can be pulled directly:

```
import wandb
api = wandb.Api()
artifact = api.artifact('nova-lora-gen-3:latest', type='model')
artifact.download('./adapter_weights')
```

Full training config is logged to each run. Seed is fixed per generation (gen * 1000).

---

9. Key Takeaways

- Zero-shot Mistral Small 3.1 achieves only 4% success on navigation — fine-tuning is essential, not optional.
- Three generations of self-improvement (300 total episodes, ~9,000 training samples) bring success rate to 84%.
- Average episode length drops from 49.1 to 14.6 steps — the model learns efficient paths, not just eventual success.
- The data flywheel works: collection success rate grows 0% -> 18% -> 43% -> 71% across generations.
- LoRA with only 0.1% trainable parameters is sufficient — no full fine-tuning needed.
- All runs, adapters, and traces are fully logged in W&B Models and Weave for reproducibility.

Created with ❤️ on Weights & Biases.

https://wandb.ai/leadgen12344-nanyang-technological-university-singapore/nova-planner/reports/NOVA-Self-Improving-Navigation-Agent-via-LoRA-Fine-Tuning--VmlldzoxNjA2ODk1MQ